

Data Quality: Principles & Industry Horror Stories



Suman

20 Feb, 2026 At 8:00 PM

Notes

Module-1

Recommended

Resource



Associated Lectures (1)



Description



Discussions

Session 4.3: Lecture Notes

In-Class Resources

Data Quality: Principles & Industry Horror Stories

Program: AIM101 -- Artificial Intelligence & Machine Learning Foundation **Session ID:** 4.3 | **Week:** 4

Prerequisites: Session 4.2 (Data Wrangling: Cleaning, Merging & GroupBy)

Mental Map

Session 4.3: Data Quality

|

+-- 1. The GIGO Principle

|

+-- IBM Watson for Oncology (\$4B+ write-down)

|

+-- Cooking analogy (rotten ingredients)

|

+-- GIGO Spectrum (4 severity levels)

|

+-- Code: Product price distortion demo

|

+-- 2. Data Quality Dimensions (6)

|

+-- Accuracy -- "Does it match reality?"

|

+-- Completeness -- "Is anything missing?"

|

+-- Consistency -- "Do related values agree?"

|

+-- Timeliness -- "Is the data current?"

|

+-- Validity -- "Does it follow the rules?"

|

+-- Uniqueness -- "Are there duplicates?"

|

+-- Code: All 6 dimension checks on patient_records

|

+-- 3. Industry Horror Stories (4)

|

+-- JPMorgan London Whale (\$6.2B Excel error)

|

+-- UK Post Office Horizon (700+ wrongful convictions)

|

+-- Target Pregnancy Prediction (ethical misuse)

|

+-- Gene Name Corruption (20%+ of genomics papers)

|

+-- Code: Gene name corruption demo

|

```

+-- 4. Data Governance
|   +-- Ownership & Stewardship
|   +-- Data Lineage
|   +-- Change Management
|   +-- Access Controls
|   +-- Code: Governance metadata + Lineage log
|
+-- 5. Documentation & Data Dictionaries
|   +-- Nutrition label analogy
|   +-- Bus factor problem
|   +-- Automated vs Enriched dictionaries
|   +-- Code: build_data_dictionary() function
|
+-- 6. Regulatory Context (GDPR, HIPAA, SOX)
|   +-- Building codes analogy
|   +-- Right to explanation
|   +-- Code: Loan decision / right to explanation demo
|
+-- 7. Connecting to ML (80/20 Reality)
    +-- ML pipeline stages
    +-- Dirty vs clean data averages
    +-- Code: Quality assessment + Mini-Project scorecard

```

The Big Picture

In Sessions 4.1 and 4.2, you learned HOW to create, select, filter, clean, merge, and aggregate data with Pandas. This session addresses the critical question that comes BEFORE all of that work: **"Is this data good enough to trust?"**

Every Pandas operation you have learned -- `.mean()`, `.groupby()`, `pd.merge()` -- produces correct results for whatever data you feed it. But if the underlying data contains errors, duplicates, inconsistencies, or missing values, those technically correct results will be factually wrong. A `.mean()` on a column with a typo (89,999 instead of 8,999) gives you a mathematically valid but completely misleading average.

This session introduces a systematic framework for evaluating data quality: six dimensions (accuracy, completeness, consistency, timeliness, validity, uniqueness), governance structures (ownership, lineage, access controls), documentation practices (data dictionaries), and the regulatory landscape (GDPR, HIPAA, SOX). You will apply all of these concepts hands-on to a messy `patient_records` dataset, producing a quantitative quality scorecard -- the kind of deliverable that data teams produce in industry before any analysis or model training begins.

Main Content

Topic 1: The GIGO Principle

GIGO = Garbage In, Garbage Out. Coined by IBM programmer George Fuechsel in the early 1960s.

- **Original meaning:** If you feed a computer incorrect input, you get incorrect output.

- **Modern ML meaning:** If you train a model on bad data, the model AMPLIFIES the errors across millions of predictions. One biased training set produces millions of biased outputs.

The cooking analogy: A world-class chef cannot turn rotten ingredients into a safe, delicious meal. No recipe or technique can save spoiled meat. Similarly, no algorithm can fix fundamentally flawed data. The analysis is ruined BEFORE you write a single line of code.

IBM Watson for Oncology: IBM invested over \$4 billion in Watson Health. Watson for Oncology was trained on synthetic and hypothetical cancer cases from a single hospital (Memorial Sloan Kettering), not on real patient outcome data at scale. The result: unsafe treatment recommendations, dissolved hospital partnerships worldwide, and IBM eventually selling Watson Health at a massive loss in 2022. The root cause was not the algorithm -- it was the training data.

The GIGO Spectrum:

Level	Severity	Example	Detection
1. Typos & Formatting	Low	"Banglore" instead of "Bangalore"	<code>.str.lower()</code> , <code>.value_counts()</code>
2. Missing Values	Medium	Empty age or diagnosis fields	<code>.isnull()</code> , <code>.isnull().sum()</code>
3. Systematic Errors	High	Sensor consistently reads 2 degrees too high	Domain knowledge, external references
4. Fabricated Data	Catastrophic	IBM Watson trained on synthetic cases as real	Data provenance investigation

Sessions 4.1 and 4.2 gave you the tools for Levels 1 and 2. This session teaches you the frameworks for Levels 3 and 4.

Code: GIGO in Action -- One Bad Data Point

```
import pandas as pd

# A product dataset with ONE data entry error
# Monitor price entered as 89,999 instead of 8,999 (extra digit)
products = pd.DataFrame({
    'product': ['Mouse', 'Keyboard', 'Monitor', 'Webcam', 'USB Cable'],
    'price': [899, 2499, 89999, 1499, 349] # ERROR: Monitor = 89,999
})

print("=== GIGO DEMO: One Bad Data Point ===\n")
print("Product data (with error):")
print(products)
print()

wrong_avg = products['price'].mean()
print(f"Average price (WITH error): ₹{wrong_avg:,.0f}\n")

# Fix the error
products.loc[products['product'] == 'Monitor', 'price'] = 8999

print("Product data (corrected):")
```

```

print(products)
print()

correct_avg = products['price'].mean()
print(f"Average price (CORRECTED): ₹{correct_avg:,.0f}\n")

print(f"Distortion factor: {wrong_avg / correct_avg:.1f}x")
print(f"One typo made the average {wrong_avg / correct_avg:.1f} times higher than reality")

```

Expected Output:

```
=== GIGO DEM0: One Bad Data Point ===
```

```
Product data (with error):
```

```

  product  price
0   Mouse    899
1  Keyboard  2499
2  Monitor 89999
3   Webcam  1499
4  USB Cable   349

```

```
Average price (WITH error): ₹19,049
```

```
Product data (corrected):
```

```

  product  price
0   Mouse    899
1  Keyboard  2499
2  Monitor  8999
3   Webcam  1499
4  USB Cable   349

```

```
Average price (CORRECTED): ₹2,849
```

```
Distortion factor: 6.7x
```

```
One typo made the average 6.7 times higher than reality!
```

One keystroke. One extra digit. The average jumped by nearly 7x. In a dataset with millions of rows, such errors are invisible without systematic quality checks.

Topic 2: The Six Data Quality Dimensions

Think of these as a complete health checkup for your data. A doctor does not check only blood pressure and declare you healthy -- similarly, checking only for missing values does not mean your data is clean.

#	Dimension	Core Question	Pandas Check
1	Accuracy	Does the data match reality?	Range checks, <code>.describe()</code>
2	Completeness	Is anything missing?	<code>.isnull().sum()</code>
3	Consistency	Do related values agree?	<code>.value_counts()</code> , <code>.nunique()</code>
4	Timeliness	Is the data current enough?	<code>pd.to_datetime()</code> , date comparisons

#	Dimension	Core Question	Pandas Check
5	Validity	Does it follow the rules?	Boolean filtering
6	Uniqueness	Are there duplicates?	<code>.duplicated()</code>

A dataset can score perfectly on one dimension and fail catastrophically on another. IBM Watson's data was arguably "complete" (no missing fields) but failed on "accuracy" (synthetic cases treated as real outcomes). You need to check ALL six.

The Messy Dataset: `patient_records`

This dataset is used for all code demos and the mini-project. It contains 25 rows with intentional quality issues across all six dimensions.

```
import pandas as pd

patient_records = pd.DataFrame({
    'patient_id': ['P001', 'P002', 'P003', 'P004', 'P005',
                  'P006', 'P007', 'P008', 'P009', 'P010',
                  'P011', 'P012', 'P013', 'P014', 'P015',
                  'P016', 'P017', 'P018', 'P019', 'P020',
                  'P021', 'P022', 'P023', 'P024', 'P025'],
    'name': ['Priya Sharma', 'Rahul Verma', 'Anita Desai', 'Vikram Patel', 'Sneha',
            'Arjun Nair', 'Meera Joshi', 'Karan Singh', 'Divya Rao', 'Amit Gupta',
            'Priya Sharma', 'Rohan Mehta', 'Neha Kulkarni', 'Suresh Reddy', 'Lal',
            'Rajesh Kumar', 'Pooja Bhat', 'Manoj Tiwari', 'Kavita Saxena', 'Deep',
            'Rahul Verma', 'Fatima Sheikh', 'Arun Pillai', 'Sonal Desai', 'Vikas'],
    'age': [28, 45, 34, -5, 62,
           38, 29, 51, None, 73,
           28, 41, 55, 160, 33,
           47, None, 36, 42, 58,
           45, 31, 67, 39, 44],
    'gender': ['F', 'M', 'F', 'M', 'F',
              'M', 'F', 'M', 'F', 'M',
              'F', 'M', 'F', 'M', 'F',
              'M', 'F', 'M', 'F', 'M',
              'M', 'F', 'M', 'F', 'M'],
    'city': ['Mumbai', 'Delhi', 'Bangalore', 'mumbai', 'Chennai',
            'Kolkata', 'Pune', 'DELHI', 'Bangalore', 'Mumbai',
            'Mumbai', 'delhi', 'Hyderabad', 'Chennai', 'Kochi',
            'Mumbai', 'Pune', 'Delhi', 'Jaipur', 'Lucknow',
            'Delhi', 'Mumbai', 'Chennai', 'bangalore', 'Pune'],
    'diagnosis': ['Diabetes', 'Hypertension', None, 'Asthma', 'Arthritis',
                 'Diabetes', 'Migraine', 'Hypertension', 'Diabetes', 'Heart Disease',
                 'Diabetes', 'Thyroid', 'Hypertension', None, 'Diabetes',
                 'Asthma', 'Anemia', 'Diabetes', 'Hypertension', 'Arthritis',
                 'Hypertension', 'Diabetes', 'Heart Disease', 'Migraine', 'Asthma'],
    'admission_date': ['2024-01-15', '2024-02-20', '2024-03-10', '2024-01-22', '2024-04-01',
                      '2024-02-14', '2024-05-01', '2024-03-18', '2024-06-12', '2024-01-30',
                      '2024-01-15', '2024-07-08', '2024-04-22', '2024-08-15', '2024-09-01',
                      '2026-12-25', '2024-05-19', '2024-11-03', '2024-10-10', '2024-06-28',
                      '2024-02-20', '2024-12-01', '2024-07-14', '2024-09-30', '2024-08-01'],
    'phone': ['9876543210', '9123456789', None, '8765432109', '9988776655',
             '9876543210', '9234567890', None, '9345678901', '9456789012',
             '9876543210', '9567890123', '9678901234', '9789012345', None,
             '9890123456', '9901234567', '8012345678', '8123456789', '8234567890']
})
```

```

        '9123456789', '8345678901', '8456789012', '8567890123', '8678901234'
    })

print("=== PATIENT RECORDS DATASET ===")
print(f"Shape: {patient_records.shape}")
print()
print(patient_records.to_string())

```

Expected Output (first rows):

```

=== PATIENT RECORDS DATASET ===
Shape: (25, 7)

```

	patient_id	name	age	gender	city	diagnosis	admission_date	phone
0	P001	Priya Sharma	28.0	F	Mumbai	Diabetes	2024-01-15	9876543210
1	P002	Rahul Verma	45.0	M	Delhi	Hypertension	2024-02-20	9123456789
2	P003	Anita Desai	34.0	F	Bangalore	None	2024-03-10	None
3	P004	Vikram Patel	-5.0	M	mumbai	Asthma	2024-01-22	8765432109
4	P005	Sneha Iyer	62.0	F	Chennai	Arthritis	2024-04-05	9988776655
...								

Quality issues planted: Age = -5 (invalid), Age = 160 (implausible), mixed-case cities ("Mumbai"/"mumbai"/"DELHI"), missing ages/diagnoses/phones, future date (2026-12-25), duplicate patients (P001/P011, P002/P021).

Dimension 1: Completeness -- "Is anything missing?"

```

print("=" * 50)
print("COMPLETENESS CHECK")
print("=" * 50)
print()

# Count missing values per column
print("Missing values per column:")
print(patient_records.isnull().sum())
print()

# Missing value percentage per column
print("Missing value percentage per column:")
print((patient_records.isnull().mean() * 100).round(1))
print()

# Overall completeness score
overall_completeness = (1 - patient_records.isnull().mean().mean()) * 100
print(f"Overall dataset completeness: {overall_completeness:.1f}%")

```

Expected Output:

```

=====
COMPLETENESS CHECK
=====

```

Missing values per column:

```
patient_id    0
name          0
age           2
gender        0
city          0
diagnosis     2
admission_date 0
phone         3
dtype: int64
```

Missing value percentage per column:

```
patient_id    0.0
name          0.0
age           8.0
gender        0.0
city          0.0
diagnosis     8.0
admission_date 0.0
phone        12.0
dtype: float64
```

Overall dataset completeness: 96.0%

Rule of thumb: Above 95% complete = generally safe. 80-95% = investigate the pattern of missingness. Below 80% = column may be unreliable.

Dimension 2: Uniqueness -- "Are there duplicates?"

```
print("=" * 50)
print("UNIQUENESS CHECK")
print("=" * 50)
print()

# Check for duplicate records based on name, age, and city
duplicates = patient_records[
    patient_records.duplicated(subset=['name', 'age', 'city'], keep=False)
]

print(f"Duplicate records found: {len(duplicates)}")
print()
print("Duplicate entries:")
print(duplicates[['patient_id', 'name', 'age', 'city', 'phone']].to_string())
print()

# Uniqueness score
total_rows = len(patient_records)
duplicate_count = patient_records.duplicated(subset=['name', 'age', 'city']).sum()
uniqueness_pct = ((total_rows - duplicate_count) / total_rows) * 100
print(f"Uniqueness score: {uniqueness_pct:.1f}%")
```

Expected Output:

```
=====
UNIQUENESS CHECK
=====
```

Duplicate records found: 4

Duplicate entries:

	patient_id	name	age	city	phone
0	P001	Priya Sharma	28.0	Mumbai	9876543210
1	P002	Rahul Verma	45.0	Delhi	9123456789
10	P011	Priya Sharma	28.0	Mumbai	9876543210
20	P021	Rahul Verma	45.0	Delhi	9123456789

Uniqueness score: 92.0%

Key: Use `keep=False` to see ALL copies (including the first), not just the extras. Default `.duplicated()` keeps the first and only marks subsequent copies.

Dimension 3: Consistency -- "Do related values agree?"

```
print("=" * 50)
print("CONSISTENCY CHECK")
print("=" * 50)
print()

# Check city column for inconsistencies
print("Unique city values (raw):")
print(patient_records['city'].value_counts())
print(f"\nTotal unique values (raw): {patient_records['city'].nunique()}")
print()

# After normalisation
normalised_cities = patient_records['city'].str.lower().str.strip()
print("City values after normalisation (.str.lower().str.strip()):")
print(normalised_cities.value_counts())
print(f"\nTotal unique values (normalised): {normalised_cities.nunique()}")
print()

# Consistency score
raw_count = patient_records['city'].nunique()
normalised_count = normalised_cities.nunique()
extra_categories = raw_count - normalised_count
print(f"Inconsistency: {extra_categories} extra categories due to case/whitespace issues")
```

Expected Output:

```
=====
CONSISTENCY CHECK
=====

Unique city values (raw):
Mumbai      4
Delhi       2
```



```
Pune      3
Chennai   3
Bangalore 2
mumbai    1
DELHI     1
delhi     1
bangalore 1
Kolkata   1
Hyderabad 1
Kochi     1
Jaipur    1
Lucknow   1
Name: city, dtype: int64
```

Total unique values (raw): 14

City values after normalisation (.str.lower().str.strip()):

```
mumbai    5
delhi     4
pune      3
chennai   3
bangalore 3
kolkata   1
hyderabad 1
kochi     1
jaipur    1
lucknow   1
Name: city, dtype: int64
```

Total unique values (normalised): 10

Inconsistency: 4 extra categories due to case/whitespace issues

The groupby test: If `.groupby('city')` would produce wrong groups, you have a consistency problem. Raw data gives 14 city groups; normalised data gives 10.

Dimension 4: Validity -- "Does it follow the rules?"

```
print("=" * 50)
print("VALIDITY CHECK")
print("=" * 50)
print()

# Check for invalid age values (negative or biologically implausible)
invalid_age = patient_records[
    (patient_records['age'] < 0) | (patient_records['age'] > 120)
]

print(f"Invalid age values (< 0 or > 120): {len(invalid_age)}")
print()
print("Invalid records:")
print(invalid_age[['patient_id', 'name', 'age']].to_string())
print()

# Validity score for age column
valid_age_count = len(patient_records) - len(invalid_age) - patient_records['age'].isnull
```

```
total_age_records = len(patient_records) - patient_records['age'].isnull().sum()
validity_pct = (valid_age_count / total_age_records) * 100
print(f"Age validity score: {validity_pct:.1f}% of non-null ages are valid")
```

Expected Output:

```
=====
VALIDITY CHECK
=====

Invalid age values (< 0 or > 120): 2

Invalid records:
  patient_id      name    age
3      P004  Vikram Patel  -5.0
13     P014  Suresh Reddy 160.0

Age validity score: 91.3% of non-null ages are valid
```

Validity vs Accuracy: A value of 25 for a 52-year-old is *valid* (plausible age) but *inaccurate* (wrong). A value of -5 is both *invalid* and *inaccurate*. Validity catches the obvious errors; accuracy catches the subtle ones.

Dimension 5: Timeliness -- "Is the data current?"

```
print("=" * 50)
print("TIMELINESS CHECK")
print("=" * 50)
print()

# Convert admission_date to datetime
patient_records['admission_date'] = pd.to_datetime(patient_records['admission_date'])

# Define a reference date
reference_date = pd.Timestamp('2024-12-31')

# Check for future dates (data entry errors)
future_dates = patient_records[patient_records['admission_date'] > reference_date]
print(f"Records with future admission dates: {len(future_dates)}")
print()
print("Future-dated records:")
print(future_dates[['patient_id', 'name', 'admission_date']].to_string())
print()

# Check for very old records
stale_threshold = pd.Timestamp('2024-01-01')
stale_records = patient_records[patient_records['admission_date'] < stale_threshold]
print(f"Records older than {stale_threshold.date()}: {len(stale_records)}")
print()

# Calculate data freshness
latest_date = patient_records['admission_date'].max()
earliest_date = patient_records['admission_date'].min()
```

```
print(f>Date range: {earliest_date.date()} to {latest_date.date()}")
print(f>Data span: {(latest_date - earliest_date).days} days")
```

Expected Output:

```
=====
TIMELINESS CHECK
=====

Records with future admission dates: 1

Future-dated records:
  patient_id      name admission_date
15      P016  Rajesh Kumar    2026-12-25

Records older than 2024-01-01: 0

Date range: 2024-01-15 to 2026-12-25
Data span: 1075 days
```

Timeliness is context-dependent: stock prices are stale after seconds; census data is updated every 10 years.

Dimension 6: Accuracy -- "Does it match reality?"

Accuracy is the hardest dimension to check with code alone. One approach is cross-validation -- checking whether duplicate patients have consistent information.

```
print("=" * 50)
print("ACCURACY CHECK (Cross-Validation)")
print("=" * 50)
print()

# Check if patients with the same name have consistent details
dup_names = patient_records[
    patient_records.duplicated(subset=['name'], keep=False)
]

print("Records with duplicate names -- check for consistency:")
print()
print(dup_names[['patient_id', 'name', 'age', 'city', 'diagnosis', 'phone']].sort_values(
print()

print("Cross-validation analysis:")
print("-- Priya Sharma (P001 & P011): Same age, city, phone -> likely true duplicate")
print("-- Rahul Verma (P002 & P021): Same age, city, phone -> likely true duplicate")
print()

# Flag obviously inaccurate values
print("Values that fail basic sanity checks (accuracy red flags):")
print(f"-- P004 (Vikram Patel): age = -5 -> impossible, likely data entry error")
print(f"-- P014 (Suresh Reddy): age = 160 -> impossible, likely digit transposition")
```

Expected Output:

```
=====
ACCURACY CHECK (Cross-Validation)
=====
```

Records with duplicate names -- check for consistency:

	patient_id	name	age	city	diagnosis	phone
0	P001	Priya Sharma	28.0	Mumbai	Diabetes	9876543210
10	P011	Priya Sharma	28.0	Mumbai	Diabetes	9876543210
1	P002	Rahul Verma	45.0	Delhi	Hypertension	9123456789
20	P021	Rahul Verma	45.0	Delhi	Hypertension	9123456789

Cross-validation analysis:

- Priya Sharma (P001 & P011): Same age, city, phone -> likely true duplicate
- Rahul Verma (P002 & P021): Same age, city, phone -> likely true duplicate

Values that fail basic sanity checks (accuracy red flags):

- P004 (Vikram Patel): age = -5 -> impossible, likely data entry error
- P014 (Suresh Reddy): age = 160 -> impossible, likely digit transposition

True accuracy verification requires checking against the real-world source (birth certificates, hospital ID systems). Pandas can catch obvious outliers and cross-validate internal records, but subtle inaccuracies require domain knowledge and external reference data.

Topic 3: Industry Horror Stories

Four real-world case studies demonstrating that data quality failures are not abstract -- they destroy companies, end careers, and ruin lives.

Case 1: JPMorgan London Whale -- \$6.2 Billion Loss (2012)

- **What happened:** A trader in JPMorgan's London office accumulated massive derivative positions. The Value at Risk (VaR) model -- maintained in Excel -- contained a copy-paste formula error: division by SUM instead of AVERAGE, halving the estimated risk.
- **The formula error:**

```
CORRECT: Risk = X / AVERAGE(A, B) = 2X / (A+B)
FLAWED:  Risk = X / SUM(A, B)      = X / (A+B)  -> Risk HALVED
```

- **Consequence:** Risk was underestimated by 2-3x. Positions grew unchecked. Total loss: *6.2billion. Over 1 billion* in regulatory fines.
- **Dimensions violated:** Accuracy (wrong formula output), Validity (output fell outside expected ranges, but nobody checked).
- **Lesson:** When your risk model runs in Excel with manual copy-paste operations, no version control, no automated testing, and no peer review -- a single wrong cell reference can cost billions.

Case 2: UK Post Office Horizon Scandal -- 700+ Wrongful Convictions (1999-2024)

- **What happened:** The Horizon IT system (by Fujitsu) generated phantom transactions -- recording money as missing from branch accounts when it was not actually missing.
- **Consequence:** Over 700 sub-postmasters were wrongfully accused of theft. 236 were criminally convicted. At least 4 people died by suicide. Families were destroyed. The Post Office knew about the bugs for years and suppressed internal reports.
- **Timeline:** 1999 (deployed) -> 2019 (High Court found Horizon contained bugs) -> 2024 (Parliament legislatively overturned all convictions).
- **Dimensions violated:** Accuracy (data did not match reality), Consistency (system records did not match actual cash).
- **Lesson:** When a system's data is treated as infallible truth without independent validation, innocent people suffer. Data accuracy is a human rights issue.

Case 3: Target Pregnancy Prediction -- Ethical Misuse (2012)

- **What happened:** Target built a model that correctly predicted customer pregnancies from purchasing patterns (unscented lotion, supplements, cotton balls). The model sent baby product coupons to a teenager's home address before her father knew she was pregnant.
- **Key insight:** The data was ACCURATE. All six technical dimensions were satisfied. The failure was in JUDGMENT -- Target did not ask "SHOULD we act on this prediction?"
- **Dimensions violated:** None technically. The failure was ethical use -- a dimension beyond the technical framework.
- **Lesson:** "Can we predict this?" is a technical question. "Should we act on this prediction?" is an ethical question. Both matter.

Case 4: Gene Name Corruption -- 20%+ of Genomics Papers (2004-ongoing)

- **What happened:** Excel auto-converts gene names to dates: SEPT2 becomes "September 2", MARCH1 becomes "March 1", DEC1 becomes "December 1". The corruption is irreversible -- once converted, the original gene name is lost.
- **Scale:** ~20% of published genomics papers with Excel supplementary files had corrupted gene data (2016 study). By 2020, it had gotten WORSE (~30%).
- **Response:** The HUGO Gene Nomenclature Committee renamed actual human genes to avoid Excel corruption (SEPT2 -> SEPTIN2 , MARCH1 -> MARCHF1).
- **Dimensions violated:** Accuracy (gene names silently changed to dates), Validity (date values in gene name columns).

Code: Gene Name Corruption Demo

```
import pandas as pd

gene_data = pd.DataFrame({
    'gene_name': ['SEPT2', 'MARCH1', 'DEC1', 'OCT4', 'FEB1', 'BRCA1', 'TP53'],
    'function': ['Cell division', 'Protein tagging', 'Transcription', 'Stem cells',
                'Cell cycle', 'Tumor suppression', 'Tumor suppression'],
    'chromosome': [2, 5, 9, 6, 1, 17, 17]
})

print("=== GENE NAME CORRUPTION DEMO ===\n")
print("In Pandas -- gene names preserved correctly:")
print(gene_data)
```

```

print(f"\nData types:")
print(gene_data.dtypes)

print("\n--- What Excel would do to these names ---")
excel_corruption = {
    'SEPT2': 'Sep-02 (date)',
    'MARCH1': 'Mar-01 (date)',
    'DEC1': 'Dec-01 (date)',
    'OCT4': 'Oct-04 (date)',
    'FEB1': 'Feb-01 (date)',
    'BRCA1': 'BRCA1 (safe)',
    'TP53': 'TP53 (safe)'
}
for gene, result in excel_corruption.items():
    print(f" {gene:10s} -> {result}")

print("\nPandas preserves ALL gene names as strings. Excel corrupts 5 out of 7.")
print("This is why bioinformaticians switched to Python/Pandas.")

```

Expected Output:

=== GENE NAME CORRUPTION DEMO ===

In Pandas -- gene names preserved correctly:

	gene_name	function	chromosome
0	SEPT2	Cell division	2
1	MARCH1	Protein tagging	5
2	DEC1	Transcription	9
3	OCT4	Stem cells	6
4	FEB1	Cell cycle	1
5	BRCA1	Tumor suppression	17
6	TP53	Tumor suppression	17

Data types:

```

gene_name    object
function     object
chromosome   int64
dtype: object

```

--- What Excel would do to these names ---

```

SEPT2      -> Sep-02 (date)
MARCH1     -> Mar-01 (date)
DEC1       -> Dec-01 (date)
OCT4       -> Oct-04 (date)
FEB1       -> Feb-01 (date)
BRCA1      -> BRCA1 (safe)
TP53       -> TP53 (safe)

```

Pandas preserves ALL gene names as strings. Excel corrupts 5 out of 7.
This is why bioinformaticians switched to Python/Pandas.

Common patterns across all 4 cases:

Pattern	JPMorgan	UK Post Office	Target	Gene Names
No Validation	VaR output not checked	System output not verified	Ethical review skipped	Excel output not verified
No Audit Trail	No version control	Logs were opaque	No human review	Original gene names lost
Human Cost	\$6.2B loss, careers ended	700+ lives ruined	Privacy violated	Scientific record corrupted
Prevention Was Cheap	Formula review (~hours)	Cash count cross-check (~minutes)	One ethics meeting	Use Pandas (~zero cost)

Topic 4: Data Governance

Data governance is the set of policies, processes, roles, and standards that define HOW data is collected, stored, maintained, and used.

The shared kitchen analogy: Think of managing a shared kitchen in a large office. Five roles map to five governance pillars:

Pillar	Kitchen Analogy	Data Governance
Ownership	Facilities manager (sets rules, budget)	Senior stakeholder who decides what data is collected and who can access it
Stewardship	Cleaning staff (daily quality checks)	Hands-on person who runs six-dimension checks regularly
Lineage	Supply chain log (where ingredients came from)	Record of where data originated and what transformations it underwent
Change Management	Need permission to rearrange appliances	Formal approval process for modifying schemas or transformation rules
Access Controls	Not everyone has the key to locked storage	Different roles get different levels of data access

Callback to horror stories: JPMorgan had no stewardship or change management for the VaR model. The Post Office had no independent stewardship to check Horizon's output. Every case had missing governance pillars.

Code: Adding Governance Metadata

```
from datetime import datetime

patient_records['data_source'] = 'hospital_admissions_system'
patient_records['last_modified'] = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
patient_records['modified_by'] = 'data_entry_team'
patient_records['version'] = 1
```

```
print("=== Data with Governance Metadata ===\n")
print(patient_records[
    ['patient_id', 'name', 'data_source', 'last_modified', 'modified_by', 'version']
].head(3).to_string())
print()

print("New metadata columns and their types:")
for col in ['data_source', 'last_modified', 'modified_by', 'version']:
    print(f" {col}: {patient_records[col].dtype}")
```

Expected Output:

=== Data with Governance Metadata ===

	patient_id	name	data_source	last_modified	modified_by
0	P001	Priya Sharma	hospital_admissions_system	2025-01-15 10:30:00	data_entry_to
1	P002	Rahul Verma	hospital_admissions_system	2025-01-15 10:30:00	data_entry_to
2	P003	Anita Desai	hospital_admissions_system	2025-01-15 10:30:00	data_entry_to

New metadata columns and their types:

```
data_source: object
last_modified: object
modified_by: object
version: int64
```

Code: Data Lineage Log

```
import pandas as pd

lineage_log = pd.DataFrame({
    'step': [1, 2, 3, 4],
    'action': [
        'Raw data imported from Hospital Admissions DB',
        'Removed duplicate patient entries',
        'Standardised city names to title case',
        'Flagged invalid ages (negative or > 120)'
    ],
    'performed_by': [
        'ETL Pipeline',
        'Data Steward',
        'Data Steward',
        'Quality Checker'
    ],
    'timestamp': [
        '2025-01-15 09:00:00',
        '2025-01-15 09:15:00',
        '2025-01-15 09:30:00',
        '2025-01-15 09:45:00'
    ],
    'rows_affected': [25, 4, 6, 2]
})

print("=== Data Lineage Log ===\n")
print(lineage_log.to_string(index=False))
print()
```



```
print(f"Total transformation steps: {len(lineage_log)}")
print(f"Total rows affected across all steps: {lineage_log['rows_affected'].sum()}")
```

Expected Output:

```
=== Data Lineage Log ===
```

step	action	performed_by	timestamp
1	Raw data imported from Hospital Admissions DB	ETL Pipeline	2025-01-15 09:00:00
2	Removed duplicate patient entries	Data Steward	2025-01-15 09:15:00
3	Standardised city names to title case	Data Steward	2025-01-15 09:30:00
4	Flagged invalid ages (negative or > 120)	Quality Checker	2025-01-15 09:45:00

```
Total transformation steps: 4
Total rows affected across all steps: 37
```

You do not need an enterprise governance platform to start. A metadata column and a lineage DataFrame are things you can build right now in any Pandas project.

Topic 5: Documentation & Data Dictionaries

The bus factor problem: The number of people who could leave before the project stalls because nobody remaining understands the data. In most data projects, the bus factor is exactly 1 -- one person who knows what the columns mean and why certain rows were excluded.

The nutrition label analogy: You would never eat unlabelled food from a stranger's kitchen. Similarly, you should never analyse an unlabelled dataset. A data dictionary tells you exactly what each "ingredient" (column) is, what values it can take, and what constraints apply.

Two levels of documentation:

Level	Source	Time	Value
Automated (Pandas)	Column names, types, null counts, unique values, samples	~1 minute	Shallow but fast
Enriched (Human)	Descriptions, business rules, valid ranges, collection methodology	~1-2 hours	Essential for understanding

Code: Automated Data Dictionary Builder

```
import pandas as pd

def build_data_dictionary(df):
    """Build a data dictionary DataFrame from any input DataFrame."""

    dictionary = pd.DataFrame({
        'column_name': df.columns,
        'data_type': [str(dtype) for dtype in df.dtypes],
        'non_null_count': [df[col].notna().sum() for col in df.columns],
        'null_count': [df[col].isna().sum() for col in df.columns],
```

```

        'null_percentage': [round(df[col].isna().mean() * 100, 1) for col in df.columns],
        'unique_values': [df[col].nunique() for col in df.columns],
        'sample_values': [str(df[col].dropna().unique()[:3].tolist()) for col in df.columns]
    })

    return dictionary

original_columns = ['patient_id', 'name', 'age', 'gender', 'city',
                    'diagnosis', 'admission_date', 'phone']

data_dict = build_data_dictionary(patient_records[original_columns])

print("=== Automated Data Dictionary ===\n")
print(data_dict.to_string(index=False))

```

Expected Output:

```
=== Automated Data Dictionary ===
```

column_name	data_type	non_null_count	null_count	null_percentage	unique_values
patient_id	object	25	0	0.0	25
name	object	25	0	0.0	23
age	float64	23	2	8.0	21
gender	object	25	0	0.0	2
city	object	25	0	0.0	14
diagnosis	object	23	2	8.0	10
admission_date	datetime64[ns]	25	0	0.0	23
phone	object	22	3	12.0	20

Code: Enriched Data Dictionary with Human Descriptions

```

descriptions = {
    'patient_id': 'Unique identifier for each patient (format: P###)',
    'name': 'Full name of the patient',
    'age': 'Patient age in years at time of admission (expected range: 0-120)',
    'gender': 'Patient gender (M = Male, F = Female)',
    'city': 'City of residence (should be title case)',
    'diagnosis': 'Primary medical diagnosis at admission',
    'admission_date': 'Date of hospital admission (format: YYYY-MM-DD)',
    'phone': '10-digit contact phone number (Indian mobile)'
}

data_dict['description'] = data_dict['column_name'].map(descriptions)

constraints = {
    'patient_id': 'Must be unique, format P followed by 3 digits',
    'name': 'Non-null, string, no special characters',
    'age': 'Integer 0-120, non-null required',
    'gender': 'One of: M, F -- no other values permitted',
    'city': 'Title case, from approved Indian city list',
    'diagnosis': 'From approved diagnosis list (ICD-10 simplified)',
    'admission_date': 'Valid date, must not be in the future',
    'phone': '10 digits, numeric only, must start with 6-9'
}

data_dict['constraints'] = data_dict['column_name'].map(constraints)

```

```
print("=== Enriched Data Dictionary ===\n")
print(data_dict[['column_name', 'data_type', 'description', 'constraints']].to_string(index=False))
```

Expected Output:

```
=== Enriched Data Dictionary ===

column_name      data_type      description
patient_id       object        Unique identifier for each patient (format: P###)
name             object        Full name of the patient
age              float64       Patient age in years at time of admission (expected...
gender           object        Patient gender (M = Male, F = Female)
city             object        City of residence (should be title case)
diagnosis        object        Primary medical diagnosis at admission
admission_date   datetime64[ns] Date of hospital admission (format: YYYY-MM-DD)
phone            object        10-digit contact phone number (Indian mobile)
```

The automated dictionary tells you what the data LOOKS like. The enriched dictionary tells you what the data MEANS. Save `build_data_dictionary()` in a utility file and reuse it in every project.

Topic 6: Regulatory Context (GDPR, HIPAA, SOX)

Data quality is not just a best practice -- for many types of data, it is a **legal requirement**.

The building code analogy: Data regulations are like building codes for construction. Building codes exist because buildings collapsed and people were killed. Data regulations exist because data failures destroyed lives and livelihoods. Neither is bureaucracy -- both are safety.

Regulation	Jurisdiction	Data Type	Key Requirement	Max Penalty
GDPR	European Union	Personal data	Data must be accurate and kept up to date	4% of global revenue or EUR 20M
HIPAA	United States	Health data	Confidentiality, integrity, availability	\$1.5M per violation category per year
SOX	United States	Financial data	Accuracy, completeness, audit trails	\$5M fine + 20 years imprisonment

The Right to Explanation (GDPR Article 22): If an automated system (including an ML model) makes a decision that significantly affects a person -- denying a loan, rejecting insurance -- that person has the RIGHT to know WHY. If the data used was WRONG, the decision may be legally invalid.

Code: Right to Explanation Demo

```
import pandas as pd

loan_applicants = pd.DataFrame({
    'applicant_id': ['A001', 'A002', 'A003'],
    'name': ['Ravi Kumar', 'Sneha Patel', 'Arjun Mehta'],
```

```

'annual_income': [500000, 350000, 800000],
'credit_score': [720, 580, 690],
'existing_loans': [1, 3, 0],
'decision': ['Approved', 'Denied', 'Approved']
})

print("=== LOAN DECISION SYSTEM ===\n")
print(loan_applicants.to_string(index=False))
print()

denied = loan_applicants[loan_applicants['decision'] == 'Denied']

print("=== Loan Denial Explanation (Right to Explanation) ===\n")

for _, row in denied.iterrows():
    print(f"Applicant: {row['name']} (ID: {row['applicant_id']})")
    print(f"Decision: {row['decision']}")
    print(f"Factors considered:")
    print(f" - Annual Income: INR {row['annual_income']:,} (threshold: INR 400,000)")
    print(f" - Credit Score: {row['credit_score']} (threshold: 650)")
    print(f" - Existing Loans: {row['existing_loans']} (max allowed: 2)")
    print()

    reasons = []
    if row['credit_score'] < 650:
        reasons.append(f"Credit score {row['credit_score']} is below threshold of 650")
    if row['existing_loans'] > 2:
        reasons.append(f"Existing loans ({row['existing_loans']}) exceed maximum of 2")
    if row['annual_income'] < 400000:
        reasons.append(f"Annual income INR {row['annual_income']:,} is below threshold of")

    for i, reason in enumerate(reasons, 1):
        label = "Primary" if i == 1 else "Secondary"
        print(f" {label} denial reason: {reason}")

print()
print("---- KEY INSIGHT ----")
print("If Sneha's income was ACTUALLY INR 3,50,000 but was entered as")
print("INR 35,000 (a data entry error -- missed a zero), the denial")
print("would be based on WRONG data. Under GDPR Article 5, the company")
print("is liable for making decisions based on inaccurate personal data.")

```

Expected Output:

```

=== LOAN DECISION SYSTEM ===

applicant_id      name  annual_income  credit_score  existing_loans  decision
A001      Ravi Kumar      500000         720             1  Approved
A002      Sneha Patel      350000         580             3   Denied
A003      Arjun Mehta      800000         690             0  Approved

=== Loan Denial Explanation (Right to Explanation) ===

Applicant: Sneha Patel (ID: A002)
Decision: Denied
Factors considered:
 - Annual Income: INR 350,000 (threshold: INR 400,000)
 - Credit Score: 580 (threshold: 650)

```

- Existing Loans: 3 (max allowed: 2)

Primary denial reason: Credit score 580 is below threshold of 650

Secondary denial reason: Existing loans (3) exceed maximum of 2

--- KEY INSIGHT ---

If Sneha's income was ACTUALLY INR 3,50,000 but was entered as INR 35,000 (a data entry error -- missed a zero), the denial would be based on WRONG data. Under GDPR Article 5, the company is liable for making decisions based on inaccurate personal data.

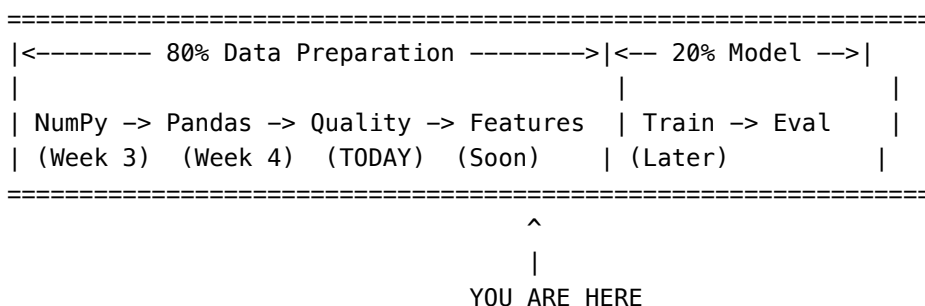
What should have caught this? A validity check (income below a reasonable threshold triggers review), a data dictionary (specifying expected income range), a data steward (cross-referencing income against supporting documents). Real data quality is multiple overlapping layers of checks.

India context: India's Digital Personal Data Protection Act (DPDPA) of 2023 has similar accuracy requirements and penalties of up to INR 250 crore per violation.

Topic 7: Connecting Data Quality to ML

The 80/20 Reality: Industry surveys consistently report that data scientists spend ~80% of their time on data preparation (collection, quality auditing, cleaning, standardisation, feature engineering) and ~20% on model development.

THE ML PIPELINE



Your NumPy and Pandas skills from Weeks 3-4 are not prerequisites you rush through to get to the "real stuff." These ARE the real stuff.

Code: Quality Assessment -- Dirty vs Clean Averages

```

import pandas as pd

print("=== STEP 1: Quality Assessment ===")
print(f"Total records: {len(patient_records)}")
print(f"Total missing values: {patient_records.isnull().sum().sum()}")
print(f"Duplicate records: {patient_records.duplicated(subset=['name', 'age', 'city']).sum}")
print(f"Quality score: {round((1 - patient_records.isnull().mean().mean()) * 100, 1)}% complete")

print("\n=== STEP 2: Danger of Skipping Quality Checks ===")
print(f"Average age (with dirty data): {patient_records['age'].mean():.1f}")

clean_ages = patient_records['age'][(patient_records['age'] >= 0) & (patient_records['age'] < 100)]
  
```

```
print(f"Average age (after removing invalid): {clean_ages.mean():.1f}")
print(f"Difference: {abs(patient_records['age'].mean() - clean_ages.mean():.1f} years!")
print("This difference could lead to completely wrong medical decisions.")
```

Expected Output:

```
=== STEP 1: Quality Assessment ===
Total records: 25
Total missing values: 7
Duplicate records: 2
Quality score: 96.5% complete

=== STEP 2: Danger of Skipping Quality Checks ===
Average age (with dirty data): 43.7
Average age (after removing invalid): 41.2
Difference: 2.5 years!
This difference could lead to completely wrong medical decisions.
```

Code: Mini-Project -- Data Quality Scorecard

This assembles all dimension checks into a single summary scorecard.

```
scorecard = pd.DataFrame({
    'dimension': ['Completeness', 'Uniqueness', 'Validity', 'Consistency', 'Timeliness',
    'check_description': [
        'Missing values across all columns',
        'Duplicate records (name + age + city)',
        'Age range (0-120), valid dates, valid gender',
        'City name standardization',
        'No future admission dates',
        'Cross-record validation'
    ],
    'issues_found': [
        patient_records.isnull().sum().sum(),
        patient_records.duplicated(subset=['name', 'age', 'city']).sum(),
        len(invalid_ages) + len(future_dates),
        patient_records['city'].nunique() - patient_records['city'].str.strip().str.title
        len(future_dates),
        len(duplicates) // 2
    ],
    'status': ['FAIL', 'FAIL', 'FAIL', 'FAIL', 'FAIL', 'FAIL']
})

total_checks = len(patient_records) * len(patient_records.columns)
total_issues = scorecard['issues_found'].sum()
overall_score = round((1 - total_issues / total_checks) * 100, 1)

print("=== DATA QUALITY SCORECARD ===")
print(scorecard.to_string(index=False))
print(f"\nOverall Data Quality Score: {overall_score}%")
print(f"\nRecommended Actions:")
print("1. Remove or merge duplicate patient records (P001/P011, P002/P021)")
print("2. Investigate and correct invalid ages (-5, 160)")
print("3. Fill missing values in diagnosis and phone columns")
```

```
print("4. Standardize city names to title case")
print("5. Verify and correct future admission dates")
```

Expected Output:

=== DATA QUALITY SCORECARD ===

dimension	check_description	issues_found	status
Completeness	Missing values across all columns	7	FAIL
Uniqueness	Duplicate records (name + age + city)	2	FAIL
Validity	Age range (0-120), valid dates, valid gender	3	FAIL
Consistency	City name standardization	3	FAIL
Timeliness	No future admission dates	1	FAIL
Accuracy	Cross-record validation	2	FAIL

Overall Data Quality Score: 91.0%

Recommended Actions:

1. Remove or merge duplicate patient records (P001/P011, P002/P021)
2. Investigate and correct invalid ages (-5, 160)
3. Fill missing values in diagnosis and phone columns
4. Standardize city names to title case
5. Verify and correct future admission dates

This scorecard is the kind of deliverable data teams produce before ANY analysis begins. Every dimension failed -- this dataset is NOT ready for analysis or model training. But now you KNOW that, and you have a prioritised action list.

Key Frameworks

Framework 1: Six Dimensions Reference Table

Dimension	Question It Answers	Pandas Code	Red Flag Example
Accuracy	Does the data match reality?	<code>df[(df['age'] < 0) (df['age'] > 120)]</code>	Age = -5, Age = 160
Completeness	Is anything missing?	<code>df.isnull().sum()</code> , <code>df.isnull().mean()</code>	12% of phone numbers missing
Consistency	Do related values agree?	<code>df['city'].value_counts()</code> , <code>.nunique()</code>	"Mumbai" and "mumbai" as separate groups
Timeliness	Is the data current enough?	<code>pd.to_datetime()</code> , date comparisons	Admission date in the future (2026)
Validity	Does it follow the rules?	Boolean filtering: <code>df[df['age'] < 0]</code>	Negative ages, impossible blood types

Dimension	Question It Answers	Pandas Code	Red Flag Example
Uniqueness	Are there duplicates?	<code>df.duplicated(subset=[...], keep=False)</code>	Same patient with two different IDs

Framework 2: Data Governance Pillars

Pillar	Definition	Real Example
Ownership	Senior stakeholder accountable for the dataset	Chief Medical Officer owns patient data
Stewardship	Hands-on person running daily quality checks	Hospital data analyst running weekly quality reports
Lineage	Record of data origin, transformations, and handlers	"Data imported from HAS on 2025-01-15, duplicates removed, cities standardised"
Change Management	Formal approval for modifying data schemas or rules	Changing age validation from 0-120 to 0-130 requires review
Access Controls	Role-based data access restrictions	Receptionist sees names/appointments; doctor sees diagnoses

Framework 3: Regulation Comparison

	GDPR (EU)	HIPAA (US)	SOX (US)
Data Type	Personal data	Health data (PHI)	Financial reporting data
Key Requirement	Accuracy + Consent	Confidentiality + Integrity	Accuracy + Audit Trails
Max Penalty	4% of global revenue or EUR 20M	\$1.5M per category per year	\$5M fine + 20 years imprisonment
Right to Explanation	Yes (Article 22)	N/A	N/A
Horror Story Link	Target (ethical use of personal data)	IBM Watson (inaccurate health recommendations)	JPMorgan (inaccurate financial reporting)

Framework 4: Quality Audit Checklist

Check completeness: `df.isnull().sum()` -- identify columns with missing values and their percentages

Check uniqueness: `df.duplicated(subset=[key_columns], keep=False)` -- find all duplicate records

Check validity: Boolean filtering for range/format rules (age 0-120, no future dates, valid categories)

Check consistency: `df['col'].value_counts()` on categorical columns -- spot case/whitespace variants

Check timeliness: Compare max date to current date; flag stale or future records

Check accuracy: `.describe()` for outliers; cross-reference duplicate records for discrepancies

Build scorecard: Assemble results into a summary DataFrame with dimension, issues found, and status

Common Errors Quick Reference

#	Error	What It Means	How to Fix It	Prevention
1	Assuming data is clean because <code>.head()</code> looks fine	<code>.head()</code> shows only the first 5 rows; quality issues may be in later rows	Always run <code>.info()</code> , <code>.describe()</code> , <code>.isnull().sum()</code> , <code>.value_counts()</code> on critical columns	Make the "five-minute exploration ritual" for every new dataset
2	Checking only for NaN but ignoring placeholder values	Placeholders like 999, -1, "N/A", "Unknown" are treated as real data by <code>.mean()</code> and <code>.groupby()</code>	Check <code>.value_counts()</code> for suspicious values; replace with NaN using <code>df.replace()</code>	After checking <code>.isnull()</code> , always check <code>.value_counts()</code> for common placeholders
3	Forgetting <code>keep=False</code> in <code>.duplicated()</code>	Default <code>.duplicated()</code> marks only the second copy; you miss the first occurrence when investigating	Use <code>df.duplicated(subset=[...], keep=False)</code> to flag ALL copies	Always use <code>keep=False</code> when investigating duplicates; use <code>drop_duplicates()</code> when deduplicating
4	Not stripping whitespace from strings	"Mumbai" and "Mumbai " (trailing space) are treated as different values in merges, groupby, and equality checks	Apply <code>.str.strip()</code> to string columns before analysis	Add <code>.str.strip()</code> to your standard cleaning pipeline for all string columns
5	Using <code>==</code> to compare with NaN instead of <code>.isna()</code>	<code>NaN != NaN</code> by design (IEEE 754). <code>df['age'] == None</code> always returns False even when values are missing	Use <code>.isna()</code> or <code>.isnull()</code> to check for missing values; never use <code>==</code> for NaN comparison	Commit to <code>.isna()</code> for missing values; <code>==</code> for specific values
6	Cleaning data in-place without keeping the original	<code>df.dropna(inplace=True)</code> destroys the original; if cleaning was too aggressive, you cannot recover	Always work on a copy: <code>df_clean = df.copy()</code>	Avoid <code>inplace=True</code> for exploratory work; keep original DataFrame untouched

#	Error	What It Means	How to Fix It	Prevention
7	Treating all missing values the same	If sicker patients are less likely to complete forms, filling with the overall mean introduces bias	Investigate the pattern of missingness; check if it correlates with other variables	Before imputing, use <code>df.groupby('otl')['col'].apply(lambda x: x.isna().mean())</code>

Key Takeaways

GIGO: No algorithm can fix fundamentally bad data. ML amplifies data problems rather than compensating for them. IBM Watson had billions of dollars and elite partnerships -- none of it mattered because the training data was flawed.

Six Dimensions: Accuracy, Completeness, Consistency, Timeliness, Validity, Uniqueness -- check ALL six systematically before any analysis. A dataset can pass one dimension and fail another.

Real Consequences: JPMorgan lost \$6.2 billion from a spreadsheet error. The UK Post Office wrongfully convicted 700+ people based on software bugs. Scientists renamed human genes because Excel corrupted them. Data quality failures destroy companies, end careers, and ruin lives.

Data Governance: Ownership, stewardship, lineage, change management, and access controls create the organizational structure for preventing quality failures. You can start today with a lineage DataFrame -- no enterprise tools required.

Documentation: Data dictionaries are non-negotiable. The automated level (Pandas introspection) takes 1 minute. The enriched level (human descriptions and constraints) takes 1-2 hours but saves countless hours of confusion later.

Regulation: GDPR, HIPAA, and SOX make data quality a legal obligation with severe penalties. The quality checks you learned today are exactly what these regulations demand.

ML Connection: 80% of ML work is data preparation. Your NumPy and Pandas skills from Weeks 3-4 are not prerequisites you rush through -- they ARE the core of the job. The quality audit scorecard is a deliverable that every data team produces before any model is trained.

Next Session Preview

Session 5.1: Data Visualization

Topic	What You Will Learn
Matplotlib Basics	Creating bar charts, line plots, histograms, scatter plots
Visual Storytelling	Choosing the right chart type for your data and audience
Customization	Labels, titles, colours, annotations
Pandas Plotting	<code>.plot()</code> method for quick visualizations

The bridge: Visualization AMPLIFIES whatever is in your data. If the data is clean, your charts reveal truth -- patterns and insights that drive good decisions. If the data is dirty, your charts create beautiful, convincing lies. The quality skills you built today are the filter that separates truthful visualizations from dangerous ones.

Same data, same chart, same code -- but one tells the truth and one tells a lie. The ONLY difference is whether a quality audit was performed first.

Vishlesan i-Hub IIT Patna x Masai School